Four Ways to Forge a Bundle My Own Verifier Calls Clean

Erik Hill · erikhill.dev · draft 2026-08-16

Abstract

I built a protocol whose premise is that a stranger can re-run my claims offline and get the same answer. An outside engineer audited it and broke it: a bundle whose headline numbers were false verified clean, and the cheapest forgery was four bytes. I merged his fix. Then I pointed my own instruments at the fixed verifier and found the same defect three more times, in places his audit did not reach. The cheapest of those is **one capital letter**.

The unifying defect is not cryptographic and not exotic. It is a check that reports success along a path where it never examined anything. I call it a **vacuous pass**.

I then stopped collecting anecdotes and measured it. Of this verifier's **112 refusal sites, 75 can be silently deleted with the entire test suite and every tamper fixture still green** — a mutation score of **0.330**. All four hand-found forgeries fall inside the surviving classes, so the measurement does not merely agree with the manual audit, it predicts it. Worse, the CI job written specifically to prove the verifier can refuse caught **zero** of the 75. Testing the refusals themselves then took the score to **0.941** — while fixing the four found defects had moved it by −0.002.

This paper documents the class, four working forgeries against a verifier that had already been hardened once, the mutation score that anticipates them, why a 137-test suite and sixteen tamper fixtures did not catch any of them, and the uncomfortable observation that my *measuring* tools lied to me six separate times during the writing of this paper — three of them inside the scripts I was using to hunt the bug, one of which scored a perfect 1.000 while measuring nothing.

1. The instrument and the claim

VAC (Verifiable Agent Claims) is an evidence-bundle format plus an offline verifier. A bundle pins every artifact by sha256, declares headline numbers in `results.summary`, and declares `results.checks` that bind those numbers to recomputable quantities in the artifacts. The verifier prints, on success:

> *proved offline: manifest schema, artifact presence + sha256, bundle closure, stated*
>
> *limitations, stamp agreement, declared results recomputed from artifacts.*

That last clause is the load-bearing one. Everything in this paper is an attack on it.

Accepted bundles are copied into a public registry (`vac/registry.py:171` copies `results.summary` verbatim) and rendered on a published page (`index.html:113`). A lie that survives the verifier is not academic — it is published.

2. The external audit

Giulio D'Erme (`GiulioDER`, author of `cca-audit`) ran a deep-tier audit after I posted an open replay request. He filed two pull requests.

His central finding: a bundle declaring `summary.verdicts: 9999` while the artifact held 3 — with `evidence/bundle.json` replaced by the four bytes `null` and **its sha256 re-pinned honestly** — exited 0 and printed `structural verification: PASS`. The hash binding was intact. The recomputation ran over nothing, found no mismatch, and passed.

Five distinct forgery paths in total, plus path traversal on an untested Python window. His second PR found something worse in kind: no `encoding=` on `read_text` / `write_text`, so **the same bytes produced opposite verdicts on a cp1252 host versus a UTF-8 host** — fatal for a protocol whose entire premise is that a stranger gets my answer offline.

Two things are worth recording because they cut the other way. First, **four of his strongest findings were refuted by my own SPEC**, not by argument — the specification had already decided those cases, and said so. His remark on that is the best sentence anyone has written about this project: *documentation that refutes an auditor is rarer than the bugs.* Second, my `RESULTS.md` byte-identity check caught mojibake in his own audit output mid-review. The instrument worked on the auditor.

3. The class: a check that cannot fail

Every finding in this paper is an instance of one shape:

> *A gate reports success along a path where it never examined the thing it claims to*
>
> *examine. The pass is real. The checking never happened.*

The `null` forgery is the pure form: recompute-and-compare where an empty recomputation compares equal to anything. But the class is much broader, and I have hit it repeatedly in work that had nothing to do with this protocol:

- A determinism test that compared two runs of the **same runtime** stayed green through a 69-diff cross-runtime divergence.
- A health endpoint returned 200 while the real code path was dead.
- A shell gate `... | grep "Tests "` matched the tally line whether tests passed or failed.
- A test that sampled an **animating** value passed against the very bug it targeted.

The common ancestor is an **absence-assertion with no liveness proof**. "Nothing bad was found" is only meaningful if you have separately established that the detector *can* fire.

4. Four forgeries against the hardened verifier

All four run against `main` at `f59fb62`, after Giulio's soundness fix. Every run is preceded by a **liveness control**: `python -m vac.verify fixtures/valid` must exit 0. Without that control a sweep proves nothing — see §7, where exactly that went wrong twice.

4.1 The lie typed as a string

`results.summary` values are compared against recomputed quantities by a walk that returns early on any leaf that is not `int` / `float`. Retype every summary number as a JSON string and nothing is ever compared. No artifact is touched. No hash is re-pinned.

```
summary: {"verdicts": 3, "fixed": 2, ...}
     ->  {"verdicts": "9999", "fixed": "9999", ...}
exit=0   structural verification: PASS
```

`SPEC.md` §2.5 explicitly blesses this — *"non-numeric, descriptive values pass through."* This is therefore a **specification** hole, not an implementation slip. The spec permitted a class of lie it had not imagined.

4.2 Deleting the check instead of breaking it

`_validate_manifest` requires only that `results.checks` be a non-empty list. **Nothing requires a listed evidence artifact to be covered by any check.** Delete the check that recomputes a number and the number moves from the strictly-bound branch to a loose branch that accepts any value appearing anywhere in the recomputed pool.

```
checks: [certlab-bundle-v1, fleet-board-v1, evalmut-run-v1, crashkit-battery-v1, modeldrift-
board-v1]
    ->  [fleet-board-v1, evalmut-run-v1, crashkit-battery-v1, modeldrift-board-v1]
exit=0   structural verification: PASS
```

Giulio's fix made *breaking* a check a named refusal. It left *deleting* one free. This is the general lesson of hardening: constraining how a check fails does not constrain whether it runs.

4.3 One capital letter

The crashkit profile weights findings by severity:

```
_CRASHKIT_WEIGHTS = {"none": 0, "low": 1, "med": 2, "high": 4, "critical": 8}     #
verify.py:548
total_w   = sum(_CRASHKIT_WEIGHTS.get(c.get("severity"), 0) ...)                  #
verify.py:594
failed_w = sum(_CRASHKIT_WEIGHTS.get(c.get("severity"), 0) ...)                   #
verify.py:596
```

`.get(sev, 0)` maps any unrecognized label to zero. Re-case the severity on the **failed** cases only and their weight vanishes from the numerator while the passing cases keep theirs. The score divides to `0.0` through the ordinary arithmetic path — it never touches a guard, an exception, or a fallback branch.

```
severities: [critical, critical, high, med, med]
        ->  [critical, Critical, high, Med, Med]      (3 re-cased, failed rows only)
    vulnerability_score declared 0.0   (honest value: 0.4545)
    evidence/eval_run.json sha256 re-pinned HONESTLY
exit=0   structural verification: PASS
```

`grep -rn severity tests/` returns **zero matches**. And `SPEC.md:306-307` states the rule plainly — *"an unknown severity weighs 0"* — so, as in §4.1, the specification wrote the hole down and called it a decision.

A security score of `0.0` on a bundle with three failed cases is then copied verbatim into the public registry.

4.4 Deleting the stamp rather than faking it

Four stamp comparisons are guarded on the artifact-side key existing (`verify.py:284, 288, 374, 383`). An issuer who deletes those keys and re-pins honestly passes all four without a single comparison executing, while the report still claims *stamp agreement*.

This one carries the cleanest discriminator in the paper, because the same fields corrupted rather than removed are refused by name:

```
WRONG VALUES (harness_commit/taskset_hash/prompt_hash = "DEADBEEF")
    exit=1   FAIL stamp-mismatch: taskset_hash: protocol 00112233445566aa, artifact DEADBEEF
             FAIL stamp-mismatch: prompt_hash:  protocol aabbccdd00112233, artifact DEADBEEF
             FAIL stamp-mismatch: harness_commit: protocol f1e2d3c, artifact DEADBEEF

KEYS DELETED (same fields, removed; sha256 re-pinned honestly)
    exit=0   structural verification: PASS
             protocol.hashes still declares all 7 pins
```

The check is demonstrably alive and fires on corruption. It is blind to absence. Two other profiles (`verify.py:639`, `verify.py:1000`) fail closed on exactly this case, so the behaviour is an asymmetry rather than a design decision.

The general form is worth stating: **a comparison guarded on both operands existing is not a check, it is a suggestion.** The party supplying one operand decides whether the comparison happens.

5. Measuring it: the verifier's own mutation score

The findings above were found by hand. That makes them anecdotes. To turn them into a measurement I applied the evalmut method to the verifier itself.

`vac/verify.py` contains **112 refusal sites** — statements of the form `failures.append("<reason>: …")`, each the sole point at which one class of bad bundle is rejected. I disabled them one at a time, replacing each with `pass`, and asked whether anything noticed. A mutant is *caught* if the 137-test suite fails, or the liveness control breaks, or any of the 16 committed tamper fixtures stops being refused.

```
112 mutants, one disabled refusal each
   37 caught      — all 37 by the unit tests
   75 SURVIVED    — undetected by the test suite AND the tamper sweep

MUTATION SCORE  37/112 = 0.330

survivors by refusal class
   26  raw-aggregate-mismatch      6  summary-mismatch
   20  artifact-unparsable         4  stamp-mismatch
   16  schema-violation            1  check-artifact-not-listed
                                   1  duplicate-artifact
                                   1  issuer-commit-mismatch
```

Two thirds of this verifier's refusals can be silently deleted and every gate stays green.

The second number is worse than the first. The 16 tamper fixtures exist as a CI job named `invalidation-liveness`, whose stated purpose is that *"the verifier must prove it can both pass and BLOCK. A gate that never fires certifies nothing."* Of the 75 mutants the unit tests missed, that job caught **zero**. Each fixture exercises exactly one refusal path, so deleting any other refusal leaves all sixteen verdicts unchanged. The gate built specifically to prove the verifier can refuse has, against this mutation set, a score of 0.

The strongest validation of the method is that it is not merely consistent with §4 — it **predicts** it. Each of the four hand-found forgeries lands in a survivor class:

§	forgery	survivor class
4.1	string-typed summary lie	`summary-mismatch` (6)
4.2	deleting the check	`check-artifact-not-listed`
4.3	capital-letter severity	`raw-aggregate-mismatch` (26)
4.4	deleting the stamp	`stamp-mismatch` (4)

Four defects took an external auditor and a day of adversarial search to find by hand. The mutation run pointed at all four categories in eleven minutes, unsupervised, and named 71 more places to look.

An honest caveat: disabling a refusal is only meaningful if some input would have reached it, so an unknown fraction of the 75 are unreachable or genuinely redundant rather than untested. That fraction is not large enough to explain a 0.330 score, and the four independently-confirmed forgeries establish that at least some survivors are live.

5.1 What hardening actually bought: nothing measurable

I then fixed all four defects in §4 and added a tamper fixture per fix, so the `invalidation-liveness` job could prove each new refusal fires. Every forgery is now refused by name; all 20 fixtures are refused against a passing live control. Re-measuring:

```
BEFORE   37/112 = 0.330
AFTER    39/119 = 0.328
```

The score did not move. Two of the four new fixtures kill a mutant; the other two kill none. And closing four holes required **seven new refusal statements, themselves untested** — `stamp-mismatch` survivors went from 4 to 8. Four bugs fixed, seven guards added, net coverage flat.

This is the most useful number in the paper, because it refutes the intuition that produced it. Fixing named bugs and pinning each with a regression fixture is exactly what a careful engineer does after an audit, and by the only measure that asks *"can this gate fail?"* it bought nothing. Coverage does not follow the defects you happened to find. The mass here is 26 surviving `raw-aggregate-mismatch` refusals concentrated in one profile's checker — none of which any audit had a reason to look at.

5.2 The measurement lied first, and scored a perfect 1.000

The first post-hardening run reported **119/119 = 1.000**.

It was false. The harness ran `pytest -x`, and the baseline was *already red* — the two tests that assert the real evalmut and crashkit bundles verify clean, which the new evidence-unchecked rule had just (correctly) broken. pytest therefore exited nonzero for every mutant, every mutant scored "caught," and the score reported a perfect suite while measuring nothing at all.

A tool built to detect checks that pass without checking produced a check that passed without checking, and the failure presented as the best possible result. Had I reported 1.000 it would have been the most flattering and most worthless number in this project.

The fix is the rule the paper already argues for, applied to itself: `tools/mutation_sweep.py` now **aborts unless the clean baseline is green**, on the grounds that a mutation score against a red baseline measures nothing. A liveness gate on the liveness instrument.

5.3 What did move it: testing the refusals, not the bugs

The survivors were never a mystery — they were a work list. I wrote **75 tests, one per surviving refusal**, organised by cluster rather than by defect.

```
                  fixing the four found bugs   0.330 -> 0.328
      testing the refusals themselves (+75)    0.330 -> 0.941
                                               112/119, 189 tests, 20/20 fixtures
```

The discipline that made them worth anything is the same one the rest of this paper is about. Each test asserts the **exact** failure list, never a substring search, so it cannot pass on a cascade it did not cause. And for most of them, disabling the target refusal makes `verify_bundle` return `[]` — the cooked bundle verifies *clean* — which proves that refusal is the **sole** guard for its defect rather than one voice in a chorus. Several tests were rewritten mid-flight when that check revealed they were leaning on a stale-hash backstop instead of the arithmetic they claimed to pin.

Seven refusals still survive, and the honest breakdown matters: **two are dead code, not untested.** `verify.py:863` is unreachable because a non-dict `flips.json` is already refused upstream by `_load_json`'s `want=dict`; `verify.py:990` is an `OSError` wrapper no bundle-shaped input can reach, since the artifact must already have been read and hashed to enter the check. Both were probed empirically rather than assumed, and no test was written to fake coverage of them.

The comparison between the two rows above is the paper's practical claim. Fixing named defects and pinning each with a regression fixture — the instinctive post-audit response — moved coverage by −0.002. Systematically asking *"can this gate fail?"* of every gate moved it by +0.611. **The defects you find are a sample; the gates you own are the population.**

A caveat on provenance, since this paper is about not trusting instruments: the 75 tests were written by six agents running concurrently against one working tree, and at least one observed a sibling mutating `vac/verify.py` underneath its own verification sweep. Their self-reported "mutation-checked" counts are therefore not independently trustworthy. The 0.941 is a single serialized run of `tools/mutation_sweep.py` against a clean tree with the liveness gate satisfied. That number is the claim; the authors' self-reports are not.

6. Why 137 tests and 16 tamper fixtures missed all of this

Every fixture encodes a forgery I had already imagined. `tamper-wrong-sha256` tests a hash that does not match its artifact. Giulio's attack re-pinned the hash *honestly*. The fixture set is a map of my own threat model, and a threat model cannot contain its own blind spot by construction.

The sharper point is a tool I already own and never turned on. **evalmut** — my own flagship — injects known defects into a system and reports which checks stayed green. That is precisely the instrument for this class. `vac-protocol` *verifies evalmut bundles*; it has never been *mutated by* evalmut. The verifier is a grader, and I never graded the grader.

That gap is where the auditor walked in.

7. The instrument lied four times while writing this

This section exists because omitting it would make the paper an instance of its own subject.

1. A tamper sweep printed *all 16 refused* while every invocation had exited 127 — a startup failure, not a refusal. The loop scored "nonzero exit" as "correctly refused." 2. Re-running the sweep months later, all 16 exited **2** — a wrong module path. Identical symptom, different cause. The live control caught it; without the control the run would have read as a clean sweep. 3. **Inside the script I wrote to hunt this bug**, a probe edit failed on a wrong filename and printed `exit=0`. That was the *unmodified* bundle passing. The tool built to find vacuous passes produced one. 4. An agent's proof-of-concept for a fifth finding ran against a **stale local `origin/main`**. After `git fetch` the premise evaporated. The finding was void. 5. The first post-hardening mutation run scored a perfect **1.000** against an already-red baseline: `pytest -x` exited nonzero for every mutant, so all 119 scored "caught." The tool built to find vacuous passes produced one, and it presented as the best possible result (§5.2). 6. Editing *this paper* to insert the §5 result, a `str.replace()` on the abstract matched nothing and returned the string unchanged. The build succeeded, the PDF regenerated, and the abstract still carried the old numbers. Python's `str.replace` cannot fail; it can only decline to do anything. The fix was to switch to an editor that **errors on no-match** — which is the entire thesis of this paper applied to a text edit.

Six instrument failures, all producing plausible output, three of them inside tools written specifically to hunt this bug. The most dangerous was not the one that broke — it was the one that returned a perfect score. The operational rules that survive:

- **Prove the instrument before the finding.** A liveness control adjacent to every sweep.
- **Re-run the ruled-out list after any fix.** A stale exclusion is indistinguishable from a real one.
- **Prefer operations that fail loudly over operations that silently no-op.** `str.replace`, `dict.get(k, default)`, `if k in a and k in b`, and `grep | true` are all the same hazard wearing different clothes: they convert "did not happen" into "fine."

8. What this implies beyond one protocol

Any eval suite, CI gate, or verifier can be audited with two questions:

1. **Can this check pass without executing?** Empty input, missing file, a process that fails to start, a pattern that matches nothing, a swallowed exception, an unknown enum value absorbed by a default. 2. **Has the detector been proven able to fire, in this run, on this host?** Not "it has a test" — a liveness control adjacent to the assertion.

A gate that has never been observed failing has not been observed working. This is the whole argument for mutation-testing eval suites rather than trusting their green.

9. Fixes

- **Unknown severity becomes a named refusal**, not weight 0 (`verify.py:594`), and `SPEC.md:306–307` changes with it. A weight table that silently absorbs unknown labels hands the issuer control of the denominator.
- **An evidence artifact not covered by any check becomes a named refusal.**
- **Non-numeric summary values are compared, not skipped** — amend `SPEC.md` §2.5 rather than patching around it.
- **Stamp keys named in `protocol.hashes` but absent from the artifact become a `stamp-mismatch`**, matching the two profiles that already fail closed.
- **A tamper fixture per fix**, so the invalidation sweep proves each new refusal can fire.
- **Wire the mutation sweep into CI as a floor.** `tools/mutation_sweep.py --floor 0.94` now exists; without a ratcheted threshold in CI the score will rot. The `invalidation-liveness` job should fail when the score drops, not merely when a fixture stops being refused.
- **Delete the two unreachable refusals** (`verify.py:863`, `:990`) rather than leave them as permanent survivors that make the denominator lie.
- **Add a tamper fixture per surviving refusal class**, starting with the four in §4.

10. Limitations

The mutation set disables refusal statements only; it does not mutate comparison operators, boundaries, or control flow, so 0.330 is an upper bound on what a fuller operator set would report. Whether a given survivor is reachable in practice is not established per-survivor — see the caveat in §5. Giulio's robustness PR is rebased and verified but **not landed** — two of its new tests build fixtures with `json.loads('{"a":' * 3000)`, which RecursionErrors inside the *decoder* on CPython 3.11 before the verifier is called. The tests require the manifest to parse while a later traversal blows the stack, and that gap's width is interpreter-dependent — the same host-dependence class the patch exists to fix. Landing it needs iterative traversals first.

Acknowledgements

Giulio D'Erme ran the audit that started this, and the four findings his review did not reach were only findable because his review taught me the shape to look for.